

LAPORAN AKHIR
IMPLEMENTASI HIGH AVAILABILITY SERVER DENGAN LOAD
BALANCING

Laporan praktikum ini disusun untuk memenuhi tugas proyek akhir kelompok Mata Kuliah
Jaringan Komputer



KELOMPOK 7

- | | | |
|---|----------------------------|--------------|
| 1 | MUHAMMAD ADIB HARYADI | (2401020076) |
| 2 | DHIYA ZARIFA PUTRI MARZUKI | (2401020075) |
| 3 | NISRINA RETNOSARI | (2401020060) |
| 4 | GIFFARI ZAKKA WALI ANDRY | (2401020088) |
| 5 | MUHAMMAD FATHIR | (2401020082) |

Dosen Pengampu:

FERDI CAHYADI, S. KOM., M. Cs.

TEKNIK INFORMATIKA
JURUSAN TEKNIK ELEKTRO DAN INFORMATIKA
FAKULTAS TEKNIK DAN TEKNOLOGI KEMARITIMAN
UNIVERSITAS MARITIM RAJA ALI HAJI

2026

KATA PENGANTAR

Puji syukur ke hadirat Tuhan Yang Maha Esa atas segala rahmat, karunia, dan penyertaan-Nya sehingga Laporan Akhir Proyek Implementasi High Availability Server dengan Load Balancing ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban pelaksanaan Project Based Learning (PjBL) pada mata kuliah Jaringan Komputer sekaligus sebagai dokumentasi menyeluruh terhadap proses perancangan, implementasi, pengujian, dan evaluasi sistem yang telah dikembangkan.

Laporan akhir ini memuat seluruh tahapan pelaksanaan proyek, dimulai dari analisis kebutuhan, perancangan topologi jaringan, implementasi lingkungan virtual menggunakan VirtualBox dan GNS3, konfigurasi layanan Apache2 pada web server, implementasi HAProxy sebagai load balancer, hingga pengujian mekanisme load balancing dan failover. Seluruh tahapan tersebut dilakukan untuk membangun sistem High Availability Server yang mampu meningkatkan ketersediaan layanan dan menjaga kontinuitas akses ketika salah satu server mengalami gangguan.

Penyusunan laporan ini tidak terlepas dari bantuan, bimbingan, dan dukungan berbagai pihak. Oleh karena itu, penulis mengucapkan terima kasih kepada dosen pengampu mata kuliah Jaringan Komputer yang telah memberikan arahan, masukan, dan bimbingan selama pelaksanaan proyek. Ucapan terima kasih juga disampaikan kepada seluruh anggota Kelompok 7 atas kerja sama, koordinasi, dan kontribusi yang telah diberikan sehingga setiap tahapan proyek dapat diselesaikan dengan baik.

Penulis menyadari bahwa laporan ini masih memiliki keterbatasan dan belum sepenuhnya sempurna. Oleh karena itu, kritik dan saran yang bersifat membangun sangat diharapkan sebagai bahan evaluasi dan perbaikan di masa yang akan datang, baik untuk pengembangan sistem maupun penyusunan laporan ilmiah.

Akhir kata, penulis berharap laporan ini dapat memberikan manfaat bagi pembaca sebagai referensi mengenai implementasi High Availability Server menggunakan HAProxy serta menjadi dokumentasi yang bermanfaat dalam pengembangan pembelajaran di bidang jaringan komputer.

Tanjungpinang, 26 Juni 2026

Kelompok 7

DAFTAR ISI

KATA PENGANTAR	ii
DAFTAR ISI.....	iii
DAFTAR TABLE.....	v
DAFTAR GAMBAR	vi
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	2
1.3 Tujuan.....	2
1.4 Manfaat.....	3
BAB II ANALISIS KEBUTUHAN.....	4
2.1 Kebutuhan Fungsional.....	4
2.2 Kebutuhan Non-Fungsional	4
2.3 Kebutuhan Perangkat Lunak dan Keras	5
2.3.1 Perangkat Lunak	5
2.3.2 Spesifikasi Hardware Minimum yang Direkomendasikan	5
BAB III DESAIN TOPOLOGI DAN IP PLAN	6
3.1 Topologi Jaringan.....	6
3.2 Perancangan Skema Pengalamatan IP dan Routing.....	8
3.2.1 Proses Kalkulasi VLSM.....	8
3.2.2 Pembagian Subnet (VLSM).....	8
3.2.3 2 IP Address Plan per Perangkat.....	9
3.2.4 Routing Table (Static Routes).....	9
3.3 Alur Komunikasi Sistem	10
BAB IV IMPLEMENTASI DAN PEMBAHASAN	11
4.1 Persiapan Lingkungan Implementasi	11
4.2 Pembuatan Virtual Machine Ubuntu Server	12
4.3 Konfigurasi Jaringan Virtual Machine	12
4.4 Integrasi Virtual Machine dengan GNS3.....	14
4.5 Hasil Implementasi Tahap Awal	15
4.6 Instalasi dan Konfigurasi Apache2	18
4.7 Konfigurasi HAProxy	20
4.8 Pengujian Load Balancing	22
4.9 Pengujian Failover.....	24
4.10 Analisis Hasil Implementasi	25
4.11 Evaluasi Sistem.....	27

BAB V PENUTUP	30
DAFTAR PUSTAKA	32

DAFTAR TABLE

Table 1 Kebutuhan Non-Fungsional Sistem	4
Table 2 Kebutuhan Perangkat Lunak.....	5
Table 3 Spesifikasi Hardware Minimum	5
Table 4 Komponen-komponen dalam topologi.	6
Table 5 Pembagian Subnet menggunakan VLSM	8
Table 6 IP Address Plan per Perangkat.....	9
Table 7 Routing Table pada router 1	9
Table 8 Fungsi virtual machine yang digunakan.	12
Table 9 hasil respon dari beberapa percobaan.	22
Table 10 Hasil Pengujian Load Balancing.....	23
Table 11 Hasil dari pengujian dengan kondisi berbeda.	24
Table 12 Hasil Pengujian Failover.....	25
Table 13 Rekapitulasi Hasil Implementasi	25
Table 14 Analisis Kesesuaian dengan Tujuan Proyek.....	26
Table 15 Evaluasi Sistem.....	27
Table 16 Evaluasi Implementasi.....	27

DAFTAR GAMBAR

Gambar 1 Topologi Jaringan High Availability	7
Gambar 2 Konfigurasi IP Address pada HAProxy	13
Gambar 3 Konfigurasi alamat IP pada Web Server 1.....	13
Gambar 4 Konfigurasi alamat IP pada Web Server 2.....	14
Gambar 5 Topologi implementasi jaringan pada GNS3.....	15
Gambar 6 Hasil traceroute dari PC1 menuju HAProxy.....	16
Gambar 7 Routing table pada Router R1	16
Gambar 8 Pengujian konektivitas dari PC1 ke seluruh perangkat.....	17
Gambar 9 Status layanan Apache2 pada Web Server 1.....	19
Gambar 10 Status layanan Apache2 pada Web Server 2.....	19
Gambar 11 Konfigurasi file <code>haproxy.cfg</code>	21
Gambar 12 Status layanan HAProxy beserta health check backend	21
Gambar 13 Hasil pengujian Round Robin menggunakan perintah <code>curl</code>	23
Gambar 14 Hasil pengujian failover ketika salah satu backend server tidak aktif.	25
Gambar 15 Ringkasan Mekanisme High Availability Setelah Implementasi	29

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi yang semakin pesat telah mendorong meningkatnya kebutuhan akan layanan web yang andal, stabil, dan selalu tersedia. Berbagai organisasi, baik di bidang pendidikan, pemerintahan, maupun industri, memanfaatkan layanan web sebagai media utama dalam penyampaian informasi, transaksi digital, penyimpanan data, hingga komunikasi dengan pengguna. Oleh karena itu, aspek *availability* atau ketersediaan layanan menjadi salah satu faktor penting yang harus diperhatikan dalam pembangunan infrastruktur jaringan.

Arsitektur layanan web konvensional yang hanya mengandalkan satu server (*single server*) memiliki kelemahan berupa *Single Point of Failure* (SPOF). Apabila server mengalami gangguan akibat kerusakan perangkat keras, kesalahan konfigurasi, pemeliharaan sistem, atau lonjakan jumlah permintaan dari pengguna, maka seluruh layanan akan berhenti dan tidak dapat diakses. Kondisi tersebut dapat menyebabkan terganggunya operasional sistem, menurunnya kualitas layanan, serta mengurangi kepercayaan pengguna terhadap sistem yang digunakan.

Salah satu pendekatan yang banyak diterapkan untuk mengatasi permasalahan tersebut adalah membangun sistem *High Availability* (HA). Konsep ini bertujuan menjaga ketersediaan layanan dengan menyediakan lebih dari satu server yang bekerja secara bersamaan sehingga ketika salah satu server mengalami gangguan, server lainnya tetap dapat melayani permintaan pengguna. Implementasi *High Availability* umumnya dipadukan dengan mekanisme *Load Balancing*, yaitu teknik pendistribusian beban kerja secara merata ke beberapa server backend agar penggunaan sumber daya menjadi lebih optimal dan tidak terjadi penumpukan beban pada satu server.

HAProxy (*High Availability Proxy*) merupakan salah satu perangkat lunak *load balancer* berbasis *open-source* yang banyak digunakan dalam implementasi *High Availability*. HAProxy mampu menangani koneksi dalam jumlah besar dengan latensi yang rendah serta menyediakan berbagai algoritma distribusi trafik, seperti *Round Robin* dan *Least Connection*. Selain itu, HAProxy dilengkapi dengan fitur *health check* yang memungkinkan sistem memantau kondisi setiap server backend secara otomatis. Apabila salah satu server mengalami kegagalan, HAProxy akan menghentikan pengiriman trafik ke

server tersebut dan secara otomatis mengalihkan seluruh permintaan ke server lain yang masih aktif (*automatic failover*) (Tanenbaum & Wetherall, 2011).

Pada proyek *Project-Based Learning* (PjBL) ini, Kelompok 7 merancang dan mengimplementasikan sistem *High Availability Server* menggunakan HAProxy sebagai *load balancer* dengan dua buah web server backend berbasis Apache2 yang berjalan pada sistem operasi Ubuntu Server 22.04 LTS. Seluruh implementasi dilakukan dalam lingkungan virtual menggunakan Oracle VirtualBox dan disimulasikan melalui GNS3 sehingga konfigurasi jaringan, proses distribusi trafik, serta mekanisme *failover* dapat diuji secara menyeluruh. Melalui proyek ini diharapkan dapat diperoleh pemahaman yang lebih mendalam mengenai penerapan teknologi *High Availability* sebagai solusi untuk meningkatkan keandalan dan ketersediaan layanan web. (HAProxy Technologies, 2024)

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah dalam proyek ini adalah sebagai berikut:

1. Bagaimana merancang dan mengimplementasikan infrastruktur *High Availability Server* menggunakan HAProxy sebagai *load balancer* dengan dua web server backend pada lingkungan simulasi GNS3?
2. Bagaimana menerapkan algoritma *Round Robin* pada HAProxy sehingga permintaan dari client dapat didistribusikan secara merata ke seluruh web server backend?
3. Bagaimana mekanisme *health check* pada HAProxy mendeteksi kegagalan salah satu web server dan melakukan proses *failover* secara otomatis?
4. Bagaimana hasil implementasi sistem *High Availability* dalam meningkatkan ketersediaan layanan dibandingkan dengan arsitektur *single server*?

1.3 Tujuan

Tujuan Umum

Merancang, mengimplementasikan, dan mengevaluasi sistem *High Availability Server* menggunakan HAProxy sebagai *load balancer* pada lingkungan simulasi GNS3 untuk meningkatkan keandalan serta ketersediaan layanan web.

Tujuan Khusus

1. Merancang topologi jaringan beserta skema pengalamatan IP menggunakan metode VLSM sesuai kebutuhan implementasi sistem.

2. Mengimplementasikan konfigurasi jaringan dan static routing sehingga seluruh perangkat dapat saling berkomunikasi dengan baik.
3. Menginstal dan mengonfigurasi Apache2 pada dua buah web server backend sebagai penyedia layanan web.
4. Mengonfigurasi HAProxy menggunakan algoritma Round Robin serta mengaktifkan fitur health check untuk memantau kondisi server backend.
5. Melakukan pengujian konektivitas, distribusi trafik (load balancing), dan mekanisme failover guna mengevaluasi keberhasilan implementasi sistem High Availability.

1.4 Manfaat

Manfaat Teoritis

1. Menambah pemahaman mengenai konsep High Availability, Load Balancing, Failover, serta penerapannya pada infrastruktur jaringan modern.
2. Memberikan gambaran mengenai implementasi HAProxy sebagai load balancer beserta mekanisme health check dalam menjaga ketersediaan layanan.
3. Menjadi referensi akademik mengenai implementasi sistem High Availability menggunakan lingkungan virtual berbasis GNS3 dan VirtualBox.

Manfaat Praktis

1. Menghasilkan dokumentasi implementasi sistem High Availability Server yang dapat dijadikan acuan dalam pengembangan infrastruktur jaringan serupa.
2. Meningkatkan kemampuan anggota kelompok dalam bidang administrasi server Linux, konfigurasi jaringan, virtualisasi, dan implementasi load balancing.
3. Memberikan contoh penerapan teknologi High Availability yang mampu meningkatkan keandalan layanan web serta meminimalkan risiko terjadinya Single Point of Failure pada suatu sistem.

BAB II

ANALISIS KEBUTUHAN

2.1 Kebutuhan Fungsional

Sistem yang akan dibangun harus mampu memenuhi kebutuhan fungsional berikut:

1. Sistem harus dapat mendistribusikan trafik HTTP dari klien secara merata ke dua web server backend (Web Server 1 dan Web Server 2) menggunakan algoritma Round Robin.
2. Sistem harus dapat mendeteksi kondisi server backend yang mengalami kegagalan secara otomatis melalui mekanisme health check yang dikonfigurasi pada HAProxy.
3. Sistem harus tetap dapat melayani permintaan klien (memberikan respons HTTP 200 OK) meskipun salah satu server backend dalam kondisi tidak aktif (down), dengan mengalihkan trafik ke server backend yang masih aktif (failover otomatis).
4. Router harus mampu meneruskan paket data antar segmen jaringan (client network, frontend network, dan backend network) menggunakan konfigurasi routing statis.
5. Setiap web server harus menampilkan halaman web yang mengidentifikasi dirinya ("Web Server 1" atau "Web Server 2") sehingga distribusi trafik dapat diverifikasi secara visual.

2.2 Kebutuhan Non-Fungsional

Table 1 Kebutuhan Non-Fungsional Sistem

No	Parameter	Target	Keterangan
1	Availability	$\geq 99\%$	Uptime layanan web saat pengujian berlangsung
2	Failover Time	≤ 10 detik	Waktu deteksi kegagalan server hingga trafik dialihkan
3	Response Time	< 500 ms	Tidak meningkat signifikan saat proses failover aktif
4	Reproducibility	100%	Konfigurasi dapat direproduksi di lingkungan GNS3 lain
5	Scalability	Modular	Arsitektur mendukung penambahan server backend

2.3 Kebutuhan Perangkat Lunak dan Keras

2.3.1 Perangkat Lunak

Table 2 Kebutuhan Perangkat Lunak

No	Perangkat Lunak	Versi	Fungsi
1	GNS3	2.2.x	Simulasi topologi jaringan (router, switch, cloud)
2	Ubuntu Server	22.04 LTS	OS untuk HAProxy, Web Server 1, dan Web Server 2
3	HAProxy	2.4.x	Load balancer dan health check backend
4	Apache2 / Nginx	2.4.x	Web server pada WS1 dan WS2
5	VirtualBox	7.x	Hypervisor untuk menjalankan VM Ubuntu
6	Draw.io	Latest	Perancangan diagram topologi jaringan
7	curl / wget	Bawaan Ubuntu	Pengujian konektivitas dan request HTTP

2.3.2 Spesifikasi Hardware Minimum yang Direkomendasikan

Table 3 Spesifikasi Hardware Minimum

Komponen	Spesifikasi Minimum	Rekomendasi
Prosesor	Intel Core i5 (4 core)	Intel Core i7 / AMD Ryzen 5 (6+ core)
RAM	8 GB	16 GB (untuk menjalankan 3–4 VM secara bersamaan)
Storage	50 GB HDD	100 GB SSD (untuk performa VM yang lebih baik)
OS Host	Windows 10 64-bit	Windows 11 / Ubuntu 22.04 Desktop
Network Adapter	1 NIC	Gigabit Ethernet

BAB III

DESAIN TOPOLOGI DAN IP PLAN

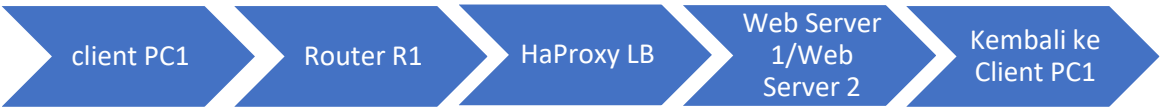
3.1 Topologi Jaringan

Infrastruktur *High Availability Server* yang dirancang pada proyek ini terdiri atas beberapa komponen jaringan yang saling terhubung dan dibagi ke dalam dua segmen jaringan, yaitu jaringan *frontend* dan jaringan *backend*. Pemisahan kedua segmen tersebut bertujuan untuk meningkatkan keamanan jaringan, mempermudah pengelolaan trafik, serta mengisolasi komunikasi antara klien dengan *load balancer* dari komunikasi antara *load balancer* dan web server backend. Pemisahan jaringan bertujuan untuk meningkatkan keamanan dan mengisolasi trafik frontend (antara klien dan load balancer) dari trafik backend (antara load balancer dan web server).

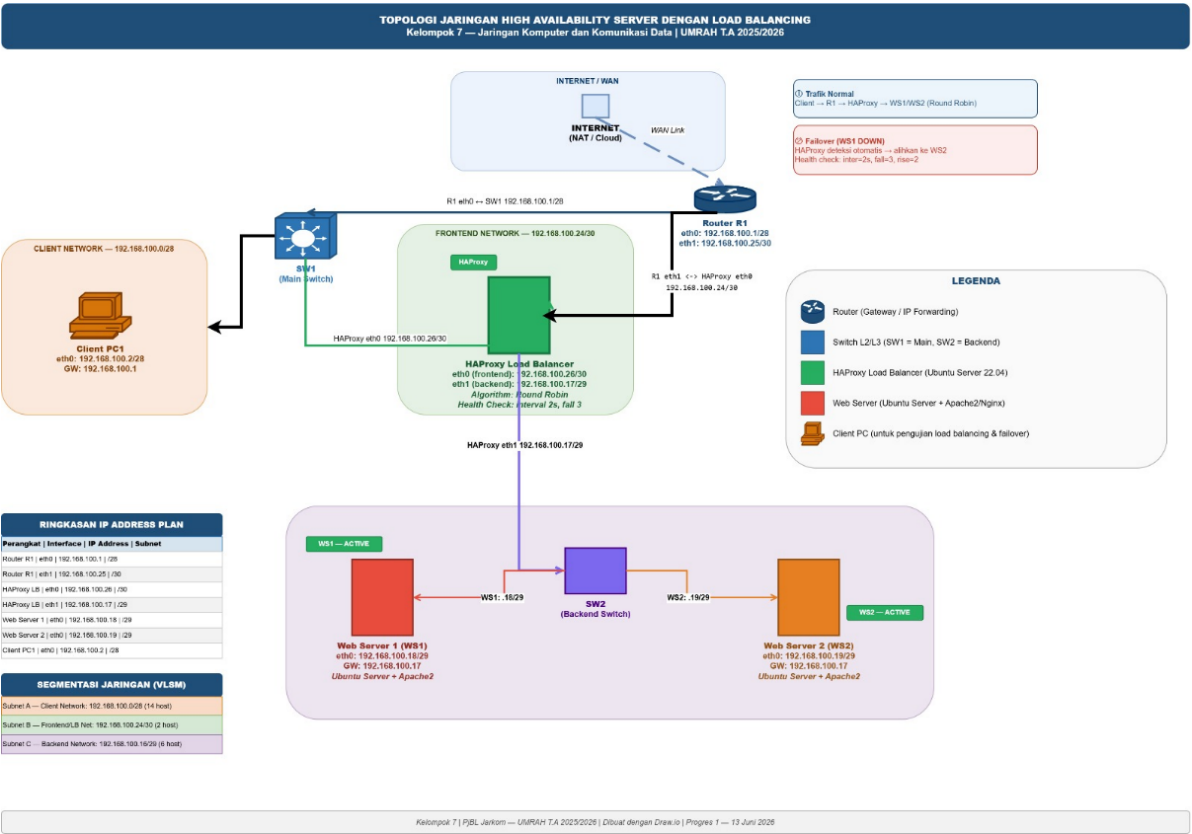
Table 4 Komponen-komponen dalam topologi.

No.	Perangkat	Fungsi
1.	Internet/Cloud (NAT)	Mewakili jaringan luar atau sumber trafik dari luar.
2.	Router R1	Gateway utama yang menghubungkan jaringan klien dengan HAProxy dan berfungsi sebagai penerus paket antar subnet (IP forwarding).
3.	Switch SW1	Switch utama pada jaringan frontend yang menghubungkan Router R1, HAProxy, dan Client PC.
4.	HAProxy Load Balancer (LB)	Komponen inti sistem HA; menerima semua request dari klien, mendistribusikannya ke WS1/WS2 menggunakan Round Robin, dan melakukan health check secara berkala.
5.	Switch SW2	Switch backend yang mengisolasi jaringan antara HAProxy dan kedua web server.
6.	Web Server 1 (WS1)	Server backend pertama yang menjalankan layanan Apache2 sebagai penyedia layanan web.
7.	Web Server 2 (WS2)	Server backend kedua yang menjalankan layanan Apache2 sebagai penyedia layanan web.
8.	Client PC1	Mesin klien untuk pengujian distribusi trafik dan failover.

Alur Trafik Normal:



Alur Trafik Saat Failover:



Gambar 1 Topologi Jaringan High Availability

3.2 Perancangan Skema Pengalamatan IP dan Routing

3.2.1 Proses Kalkulasi VLSM

Perancangan skema pengalamatan IP pada penelitian ini menggunakan *address space* 192.168.100.0/24. Pembagian alamat dilakukan menggunakan metode Variable Length Subnet Mask (VLSM) agar alokasi alamat IP menjadi lebih efisien sesuai kebutuhan jumlah host pada masing-masing subnet. (Internet Engineering Task Force, 1993).

Langkah-langkah kalkulasi:

1. Identifikasi kebutuhan host per subnet:
 - Subnet A (Client Network): 10 host (klien + gateway router)
 - Subnet C (Backend Network): 3 host (HAProxy backend + WS1 + WS2)
 - Subnet B (Frontend/LB Network): 2 host (Router R1 eth1 + HAProxy eth0)
2. Selanjutnya kebutuhan host diurutkan berdasarkan jumlah host terbesar menuju yang terkecil sehingga proses pembagian subnet menjadi lebih efisien:
 - 10 host $\rightarrow$ /28 ($2^4 = 16$ alamat, 14 usable)
 - 3 host $\rightarrow$ /29 ($2^3 = 8$ alamat, 6 usable)
 - 2 host $\rightarrow$ /30 ($2^2 = 4$ alamat, 2 usable)
3. Alokasikan secara berurutan dengan menjaga batas subnet yang benar.

3.2.2 Pembagian Subnet (VLSM)

Table 5 Pembagian Subnet menggunakan VLSM

No	Nama Subnet	Keb. Host	Network Address	Subnet Mask	CIDR	Broadcast	Host Pertama	Host Terakhir	Host Tersedia
1	Client Network	10	192.168.100.0	255.255.255.240	/28	192.168.100.15	192.168.100.1	192.168.100.14	14
2	Backend Network	3	192.168.100.16	255.255.255.248	/29	192.168.100.23	192.168.100.17	192.168.100.22	6

3	Frontend/LB Net	2	192.168.100.24	255.255.252.52	/30	192.168.100.27	192.168.100.25	192.168.100.26	2
---	-----------------	---	----------------	----------------	-----	----------------	----------------	----------------	---

3.2.3 2 IP Address Plan per Perangkat

Table 6 IP Address Plan per Perangkat

No	Perangkat	Interface	IP Address	Subnet Mask	Gateway	Keterangan
1	Router R1	eth0	192.168.100.1	255.255.255.240	—	Gateway Client Network (/28)
2	Router R1	eth1	192.168.100.25	255.255.255.252	—	Gateway Frontend/LB Net (/30)
3	HAProxy LB	eth0	192.168.100.26	255.255.255.252	192.168.100.25	Frontend, terhubung ke Router R1
4	HAProxy LB	eth1	192.168.100.17	255.255.255.248	—	Backend, gateway untuk WS1 & WS2
5	Web Server 1	eth0	192.168.100.18	255.255.255.248	192.168.100.17	Backend Network (/29)
6	Web Server 2	eth0	192.168.100.19	255.255.255.248	192.168.100.17	Backend Network (/29)
7	Client PC1	eth0	192.168.100.2	255.255.255.240	192.168.100.1	Client Network (/28)

3.2.4 Routing Table (Static Routes)

Table 7 Routing Table pada router 1

Destination	Mask	Next Hop	Interface	Keterangan
192.168.100.0	255.255.255.240	-	eth0	Client Network (directly connected)

192.168.100.24	255.255.255.252	-	eth1	Frontend/LB Net (directly connected)
192.168.100.16	255.255.255.248	192.168.100.26	eth1	Backend Network via HAProxy
0.0.0.0	0.0.0.0	NAT/Cloud	eth0	Default route ke Internet

3.3 Alur Komunikasi Sistem

Proses komunikasi pada sistem dimulai ketika client (PC1) mengirimkan permintaan HTTP menuju alamat IP frontend HAProxy. Permintaan tersebut terlebih dahulu diteruskan menuju Router R1 yang berperan sebagai gateway utama pada jaringan frontend. Router kemudian mengarahkan paket ke HAProxy melalui jaringan frontend berdasarkan konfigurasi static routing yang telah ditetapkan.

Setelah menerima permintaan dari client, HAProxy bertindak sebagai *load balancer* yang akan memilih salah satu web server backend menggunakan algoritma Round Robin. Pada kondisi normal, permintaan akan didistribusikan secara bergantian ke Web Server 1 dan Web Server 2 sehingga beban kerja dapat dibagi secara merata.

Setelah backend memproses permintaan, respons HTTP dikirimkan kembali menuju HAProxy, kemudian diteruskan kepada client sehingga proses komunikasi berlangsung secara transparan.

Selain mendistribusikan trafik, HAProxy secara berkala melakukan *health check* terhadap kedua backend menggunakan parameter yang telah dikonfigurasi. Apabila salah satu backend gagal memberikan respons sesuai batas yang telah ditentukan, HAProxy akan menandai server tersebut dalam kondisi *DOWN* dan secara otomatis mengalihkan seluruh permintaan baru ke backend yang masih aktif. Mekanisme ini memastikan layanan tetap tersedia meskipun terjadi gangguan pada salah satu server.

BAB IV

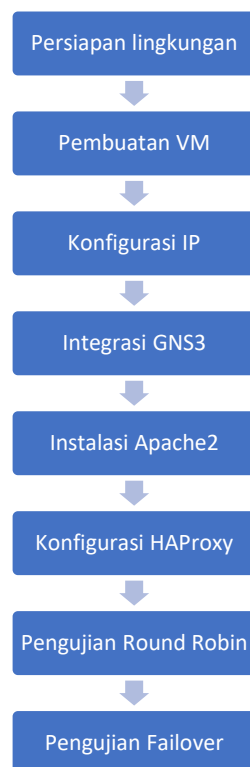
IMPLEMENTASI DAN PEMBAHASAN

4.1 Persiapan Lingkungan Implementasi

Tahap implementasi diawali dengan menyiapkan seluruh lingkungan kerja yang akan digunakan dalam pembangunan sistem High Availability Server. Implementasi dilakukan pada komputer host yang menjalankan sistem operasi Windows dengan memanfaatkan VirtualBox sebagai hypervisor untuk menjalankan beberapa mesin virtual Ubuntu Server serta GNS3 sebagai simulator jaringan.

Perangkat lunak yang digunakan meliputi VirtualBox versi 7.x, Ubuntu Server 22.04 LTS, GNS3 versi 2.2.x, HAProxy sebagai load balancer, Apache2 sebagai web server, serta Cisco IOS Router yang dijalankan melalui GNS3. Seluruh perangkat lunak tersebut dipilih karena bersifat stabil, banyak digunakan pada lingkungan industri, serta mendukung implementasi High Availability secara optimal.

Sebelum konfigurasi dilakukan, setiap virtual machine dipastikan dapat berjalan dengan baik dan memiliki spesifikasi perangkat keras virtual yang memadai, seperti RAM, kapasitas penyimpanan, serta jumlah adapter jaringan sesuai kebutuhan topologi yang telah dirancang pada Bab III.



4.2 Pembuatan Virtual Machine Ubuntu Server

Implementasi dimulai dengan membuat tiga buah Virtual Machine menggunakan Oracle VirtualBox. Ketiga virtual machine tersebut masing-masing memiliki fungsi yang berbeda, yaitu sebagai server load balancer dan dua buah web server backend. Adapun pembagian fungsi virtual machine adalah sebagai berikut.

Table 8 Fungsi virtual machine yang digunakan.

No	Nama Virtual Machine	Fungsi
1	HAProxy	Load Balancer
2	Web Server 1	Backend Server 1
3	Web Server 2	Backend Server 2

Seluruh virtual machine menggunakan sistem operasi Ubuntu Server 22.04 LTS agar penggunaan sumber daya lebih ringan dibandingkan sistem operasi berbasis desktop. Setelah proses instalasi selesai, masing-masing virtual machine diberikan hostname sesuai fungsinya untuk mempermudah proses administrasi sistem.

Tahapan berikutnya adalah melakukan pembaruan (update) repository sistem agar seluruh paket yang akan diinstal menggunakan versi terbaru.

```
sudo apt update
```

```
sudo apt upgrade -y
```

Perintah `sudo apt update` digunakan untuk memperbarui informasi paket yang tersedia pada repositori Ubuntu sehingga sistem mengetahui versi perangkat lunak terbaru yang dapat diinstal. Selanjutnya, `sudo apt upgrade -y` digunakan untuk memperbarui seluruh paket yang telah terpasang ke versi terbaru secara otomatis. Pembaruan sistem dilakukan untuk memastikan setiap virtual machine berada pada kondisi yang stabil, memperoleh pembaruan keamanan (*security update*), serta meminimalkan kemungkinan terjadinya konflik paket saat proses instalasi Apache2 maupun HAProxy pada tahap berikutnya.

4.3 Konfigurasi Jaringan Virtual Machine

Setelah seluruh virtual machine berhasil dibuat, tahap berikutnya adalah melakukan konfigurasi jaringan sesuai dengan rancangan IP Address Plan yang telah disusun pada Bab III. HAProxy dikonfigurasi menggunakan dua buah network interface. Interface pertama digunakan sebagai frontend yang menerima permintaan dari client melalui router, sedangkan interface kedua digunakan sebagai backend yang terhubung langsung dengan kedua web

server. Sementara itu, masing-masing web server hanya menggunakan satu interface jaringan yang terhubung pada jaringan backend.

```
timeout client 50000
timeout server 50000
errorfile 400 /etc/haproxy/errors/400.http
errorfile 403 /etc/haproxy/errors/403.http
errorfile 408 /etc/haproxy/errors/408.http
errorfile 500 /etc/haproxy/errors/500.http
errorfile 502 /etc/haproxy/errors/502.http
errorfile 503 /etc/haproxy/errors/503.http
errorfile 504 /etc/haproxy/errors/504.http

frontend web_front
    bind *:80
    default_backend web_servers

backend web_servers
    balance roundrobin
    option httpchk GET /
    server webserver1 192.168.56.11:80 check
    server webserver2 192.168.56.12:80 check

adib@haproxy:~$ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:06:63:26 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.10/24 brd 192.168.56.255 scope global enp0s3
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe06:6326/64 scope link
        valid_lft forever preferred_lft forever
adib@haproxy:~$
```

Gambar 2 Konfigurasi IP Address pada HAProxy

Konfigurasi alamat IP dilakukan secara statis sehingga setiap perangkat selalu memperoleh alamat yang sama setiap kali sistem dijalankan. Pendekatan ini dipilih untuk mempermudah proses routing serta konfigurasi HAProxy.

```
System information as of Fri Jun 26 12:39:31 PM UTC 2026

System load:  0.01      Processes:           99
Usage of /:   34.1% of 9.74GB    Users logged in:    0
Memory usage: 22%      IPv4 address for enp0s3: 192.168.56.11
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

238 updates can be applied immediately.
178 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection
or proxy settings

Last login: Fri Jun 26 12:02:37 UTC 2026 on tty1
adib@webserver1:~$ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:b8:3b:c2 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.11/24 brd 192.168.56.255 scope global enp0s3
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:feb8:3bc2/64 scope link
        valid_lft forever preferred_lft forever
adib@webserver1:~$
```

Gambar 3 Konfigurasi alamat IP pada Web Server 1

```

System information as of Fri Jun 26 12:41:34 PM UTC 2026

System load:  0.08      Processes:           100
Usage of /:   34.1% of 9.74GB  Users logged in:    0
Memory usage: 22%      IPv4 address for enp0s3: 192.168.56.12
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

238 updates can be applied immediately.
178 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection
or proxy settings

Last login: Fri Jun 26 08:26:04 UTC 2026 on tty1
adib@webserver2:~$ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:b2:60:17 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.12/24 brd 192.168.56.255 scope global enp0s3
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:feb2:6017/64 scope link
        valid_lft forever preferred_lft forever
adib@webserver2:~$

```

Gambar 4 Konfigurasi alamat IP pada Web Server 2

Konfigurasi alamat IP dilakukan secara statis dengan mengacu pada *IP Address Plan* yang telah dirancang pada Bab III. Penggunaan alamat IP statis dipilih karena setiap perangkat pada sistem memiliki fungsi yang tetap sehingga tidak memerlukan alokasi alamat secara dinamis menggunakan DHCP. Selain mempermudah administrasi jaringan, pendekatan ini juga memudahkan proses konfigurasi routing, pengaturan backend pada HAProxy, serta pelaksanaan pengujian konektivitas selama implementasi berlangsung.

Setelah konfigurasi selesai dilakukan, konektivitas antar perangkat diuji menggunakan utilitas ping untuk memastikan seluruh perangkat telah berada pada subnet yang benar dan dapat saling berkomunikasi. Hasil pengujian menunjukkan bahwa seluruh virtual machine berhasil saling terhubung tanpa mengalami packet loss sehingga implementasi dapat dilanjutkan ke tahap berikutnya.

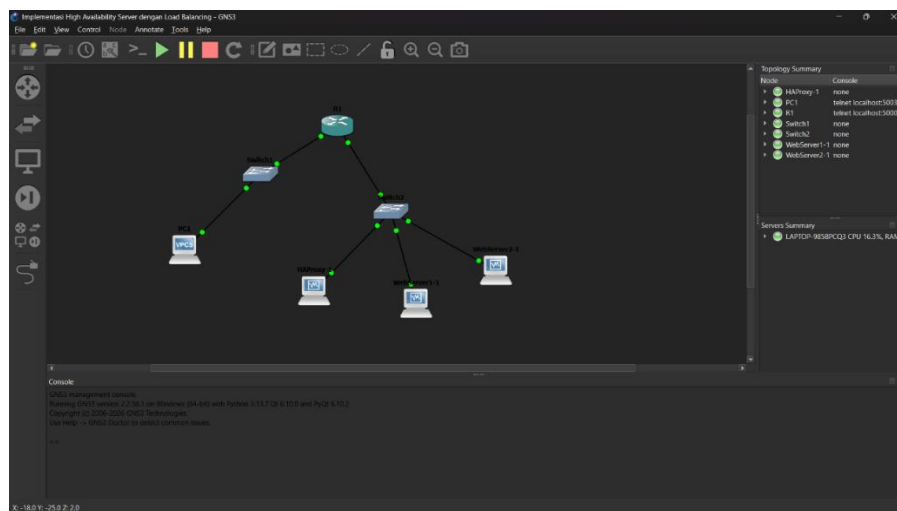
Pengujian konektivitas merupakan tahapan penting sebelum sistem memasuki proses implementasi layanan. Seluruh perangkat harus dapat saling berkomunikasi agar proses distribusi trafik nantinya dapat berjalan tanpa kendala. Berdasarkan hasil pengujian, seluruh paket berhasil dikirim dan diterima tanpa mengalami *packet loss*, sehingga dapat disimpulkan bahwa konfigurasi pengalamatan IP serta konektivitas dasar jaringan telah sesuai dengan rancangan sistem.

4.4 Integrasi Virtual Machine dengan GNS3

Setelah konfigurasi jaringan pada virtual machine selesai dilakukan, seluruh virtual machine kemudian diintegrasikan ke dalam lingkungan simulasi GNS3. Integrasi dilakukan

menggunakan VirtualBox Guest sehingga setiap virtual machine dapat berfungsi sebagai node pada topologi jaringan yang telah dirancang sebelumnya.

Pada tahap ini juga ditambahkan beberapa perangkat jaringan lain seperti Cisco Router, Ethernet Switch, Cloud NAT, dan VPCS sebagai client. Seluruh perangkat kemudian dihubungkan menggunakan koneksi Ethernet sesuai dengan desain topologi sehingga terbentuk jalur komunikasi antara client, router, HAProxy, serta kedua web server backend.



Gambar 5 Topologi implementasi jaringan pada GNS3

Integrasi antara Oracle VirtualBox dan GNS3 memungkinkan setiap virtual machine berperan sebagai perangkat jaringan nyata di dalam simulator. Dengan pendekatan ini, seluruh komunikasi antarperangkat dapat diamati secara langsung sehingga proses konfigurasi, pengujian routing, maupun analisis lalu lintas jaringan dapat dilakukan secara lebih realistis dibandingkan hanya menggunakan simulator tanpa sistem operasi server.

Setelah seluruh koneksi selesai dibuat, dilakukan pengujian awal menggunakan perintah ping dari client menuju router, HAProxy, Web Server 1, dan Web Server 2. Seluruh perangkat berhasil memberikan balasan (reply), yang menunjukkan bahwa konfigurasi routing dasar telah berjalan dengan baik.

Keberhasilan pengujian konektivitas menunjukkan bahwa konfigurasi topologi, pengalamatan IP, serta *static routing* telah diterapkan dengan benar. Tahap ini menjadi dasar bagi implementasi layanan web dan mekanisme *High Availability* pada tahap selanjutnya.

4.5 Hasil Implementasi Tahap Awal

Berdasarkan tahapan implementasi yang telah dilakukan, seluruh lingkungan kerja berhasil dipersiapkan sesuai dengan rancangan sistem pada Bab III. Tiga buah virtual machine Ubuntu Server berhasil dibuat menggunakan Oracle VirtualBox dengan pembagian

fungsi sebagai Load Balancer (HAProxy), Web Server 1, dan Web Server 2. Seluruh virtual machine dapat dijalankan dengan baik tanpa mengalami kendala yang berarti selama proses instalasi.

```
PC1> trace 192.168.56.10
Trace to 192.168.56.10, 8 hops max, press Ctrl+C to stop
 1  192.168.10.1    10.279 ms  10.178 ms  10.180 ms
 2  *192.168.56.10  20.362 ms (ICMP type:3, code:3, Destination port unreachable)
```

Gambar 6 Hasil traceroute dari PC1 menuju HAProxy

Konfigurasi alamat IP statis pada masing-masing virtual machine juga telah berhasil diterapkan sesuai dengan IP Address Plan yang telah dirancang sebelumnya. Penggunaan alamat IP statis memberikan kemudahan dalam proses administrasi jaringan serta memastikan setiap perangkat memiliki identitas yang tetap selama proses implementasi berlangsung.

Tahap berikutnya yaitu integrasi virtual machine ke dalam lingkungan simulasi GNS3 berhasil dilakukan. Seluruh perangkat jaringan, meliputi Router Cisco, Ethernet Switch, Cloud NAT, HAProxy, kedua Web Server, serta VPCS sebagai client, telah terhubung sesuai dengan topologi yang dirancang.

```
C    192.168.10.0/24 is directly connected, FastEthernet0/0
C    192.168.56.0/24 is directly connected, FastEthernet1/0
R1#
```

Gambar 7 Routing table pada Router R1

Pengujian konektivitas menggunakan perintah ping menunjukkan bahwa seluruh perangkat dapat saling berkomunikasi dengan baik tanpa mengalami packet loss. Hal tersebut menunjukkan bahwa konfigurasi jaringan dasar, pengalamatan IP, serta routing awal telah berjalan sesuai dengan perencanaan sehingga sistem siap untuk memasuki tahap implementasi layanan pada progres berikutnya.

```
Build time: Apr 10 2019 02:42:20
Copyright (c) 2007-2014, Paul Meng (mirnshi@gmail.com)
All rights reserved.

VPCS is free software, distributed under the terms of the "BSD" licence.
Source code and license can be found at vpcs.sf.net.
For more information, please visit wiki.freecode.com.cn.

Press '?' to get help.

Executing the startup file

PC1> ip 192.168.10.2 255.255.255.0 192.168.10.1
Checking for duplicate address...
PC1 : 192.168.10.2 255.255.255.0 gateway 192.168.10.1

PC1> ping 192.168.10.1
84 bytes from 192.168.10.1 icmp_seq=1 ttl=255 time=10.244 ms
84 bytes from 192.168.10.1 icmp_seq=2 ttl=255 time=6.610 ms
84 bytes from 192.168.10.1 icmp_seq=3 ttl=255 time=6.704 ms
84 bytes from 192.168.10.1 icmp_seq=4 ttl=255 time=5.600 ms
84 bytes from 192.168.10.1 icmp_seq=5 ttl=255 time=5.597 ms

PC1> ping 192.168.56.10
84 bytes from 192.168.56.10 icmp_seq=1 ttl=63 time=18.055 ms
84 bytes from 192.168.56.10 icmp_seq=2 ttl=63 time=16.360 ms
84 bytes from 192.168.56.10 icmp_seq=3 ttl=63 time=16.526 ms
84 bytes from 192.168.56.10 icmp_seq=4 ttl=63 time=16.532 ms
84 bytes from 192.168.56.10 icmp_seq=5 ttl=63 time=17.998 ms

PC1> ping 192.168.56.11
84 bytes from 192.168.56.11 icmp_seq=1 ttl=63 time=20.065 ms
84 bytes from 192.168.56.11 icmp_seq=2 ttl=63 time=16.485 ms
84 bytes from 192.168.56.11 icmp_seq=3 ttl=63 time=17.523 ms
84 bytes from 192.168.56.11 icmp_seq=4 ttl=63 time=16.900 ms
84 bytes from 192.168.56.11 icmp_seq=5 ttl=63 time=17.079 ms

PC1> ping 192.168.56.12
84 bytes from 192.168.56.12 icmp_seq=1 ttl=63 time=17.797 ms
84 bytes from 192.168.56.12 icmp_seq=2 ttl=63 time=15.235 ms
84 bytes from 192.168.56.12 icmp_seq=3 ttl=63 time=16.524 ms
84 bytes from 192.168.56.12 icmp_seq=4 ttl=63 time=17.222 ms
84 bytes from 192.168.56.12 icmp_seq=5 ttl=63 time=15.951 ms

PC1> curl http://192.168.56.10
Bad command: "curl http://192.168.56.10". Use ? for help.

PC1> 
```

Gambar 8 Pengujian konektivitas dari PC1 ke seluruh perangkat

Selain menggunakan perintah ping, dilakukan pula pengujian menggunakan traceroute dan pemeriksaan routing table pada Router R1. Hasil traceroute menunjukkan bahwa jalur komunikasi dari client menuju HAProxy telah melewati rute yang sesuai dengan konfigurasi jaringan. Sementara itu, routing table memperlihatkan bahwa seluruh subnet telah dikenali oleh router sehingga proses penerusan paket (packet forwarding) dapat berlangsung dengan baik.

Berdasarkan seluruh pengujian pada tahap implementasi awal, dapat disimpulkan bahwa infrastruktur jaringan telah berhasil dibangun sesuai dengan rancangan. Seluruh perangkat mampu saling berkomunikasi dan siap digunakan sebagai dasar implementasi layanan Apache2 dan HAProxy pada tahap berikutnya.

4.6 Instalasi dan Konfigurasi Apache2

Setelah infrastruktur jaringan berhasil dibangun pada tahap sebelumnya, implementasi dilanjutkan dengan memasang layanan web server pada masing-masing backend server. Layanan yang digunakan adalah Apache HTTP Server (Apache2) karena merupakan salah satu web server yang bersifat open source, ringan, stabil, dan banyak digunakan pada implementasi layanan berbasis Linux.

Instalasi Apache2 dilakukan pada Web Server 1 dan Web Server 2 menggunakan manajer paket APT pada sistem operasi Ubuntu Server. Setelah proses instalasi selesai, layanan Apache dijalankan serta diatur agar aktif secara otomatis ketika sistem dinyalakan kembali (*boot*). Untuk memastikan layanan berjalan dengan baik, dilakukan pemeriksaan status menggunakan perintah berikut.

```
sudo systemctl status apache2
```

Perintah `systemctl status apache2` digunakan untuk memastikan bahwa layanan Apache2 telah berjalan dengan baik pada sistem operasi Ubuntu Server. Informasi yang ditampilkan meliputi status layanan, waktu mulai (*uptime*), proses yang sedang berjalan, serta kondisi layanan secara keseluruhan. Status *active (running)* menunjukkan bahwa web server telah aktif dan siap menerima permintaan HTTP dari HAProxy. Hasil pemeriksaan menunjukkan bahwa layanan Apache2 pada kedua web server berada pada status *active (running)*. Hal ini menandakan bahwa web server telah berhasil diinstal dan siap menerima permintaan HTTP dari HAProxy.

Selanjutnya, masing-masing web server diberikan halaman web sederhana yang berbeda sebagai identitas server sehingga proses distribusi trafik nantinya dapat diamati dengan mudah. Web Server 1 menampilkan identitas "WEB SERVER 1", sedangkan Web Server 2 menampilkan identitas "WEB SERVER 2".


```

Last login: Fri Jun 26 12:02:37 UTC 2026 on tty1
adib@webserver1:~$ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:b8:3b:c2 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.11/24 brd 192.168.56.255 scope global enp0s3
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:feb8:3bc2/64 scope link
        valid_lft forever preferred_lft forever
adib@webserver1:~$ sudo systemctl status apache2
[sudo] password for adib:
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/lib/systemd/system/apache2.service; enabled; vendor preset: enabled)
   Active: active (running) since Fri 2026-06-26 12:07:19 UTC; 37min ago
     Docs: https://httpd.apache.org/docs/2.4/
   Process: 604 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
   Main PID: 705 (apache2)
     Tasks: 55 (limit: 1011)
    Memory: 9.3M
       CPU: 1.138s
   CGroup: /system.slice/apache2.service
           └─705 /usr/sbin/apache2 -k start
             716 /usr/sbin/apache2 -k start
             717 /usr/sbin/apache2 -k start

Jun 26 12:07:19 webserver1 systemd[1]: Starting The Apache HTTP Server...
Jun 26 12:07:19 webserver1 apachectl[671]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, please add the 'ServerName' directive to the configuration file to avoid this error
Jun 26 12:07:19 webserver1 systemd[1]: Started The Apache HTTP Server.
lines 1-17/17 (END)

```

Gambar 9 Status layanan Apache2 pada Web Server 1

```

Last login: Fri Jun 26 08:26:04 UTC 2026 on tty1
adib@webserver2:~$ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:b2:60:17 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.12/24 brd 192.168.56.255 scope global enp0s3
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:feb2:6017/64 scope link
        valid_lft forever preferred_lft forever
adib@webserver2:~$ sudo systemctl status apache2
[sudo] password for adib:
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/lib/systemd/system/apache2.service; enabled; vendor preset: enabled)
   Active: active (running) since Fri 2026-06-26 09:06:38 UTC; 3h 40min ago
     Docs: https://httpd.apache.org/docs/2.4/
   Process: 605 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
   Main PID: 833 (apache2)
     Tasks: 55 (limit: 1011)
    Memory: 9.8M
       CPU: 6.576s
   CGroup: /system.slice/apache2.service
           └─833 /usr/sbin/apache2 -k start
             835 /usr/sbin/apache2 -k start
             836 /usr/sbin/apache2 -k start

Jun 26 09:06:28 webserver2 systemd[1]: Starting The Apache HTTP Server...
Jun 26 09:06:38 webserver2 apachectl[665]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, please add the 'ServerName' directive to the configuration file to avoid this error
Jun 26 09:06:38 webserver2 systemd[1]: Started The Apache HTTP Server.
lines 1-17/17 (END)

```

Gambar 10 Status layanan Apache2 pada Web Server 2

Perintah `systemctl status apache2` digunakan untuk memastikan bahwa layanan Apache2 telah berjalan dengan baik pada sistem operasi Ubuntu Server. Informasi yang ditampilkan meliputi status layanan, waktu mulai (*uptime*), proses yang sedang berjalan, serta kondisi layanan secara keseluruhan. Status `active (running)` menunjukkan bahwa web server telah aktif dan siap menerima permintaan HTTP dari HAProxy.

```
<!-- Web Server 1 -->
```

```
<h1>WEB SERVER 1</h1>
```

```
<!-- Web Server 2 -->
<h1>WEB SERVER 2</h1>
```

Potongan kode HTML di atas digunakan sebagai halaman utama (*index.html*) pada masing-masing web server. Meskipun sederhana, halaman ini sangat membantu dalam proses pengujian karena memudahkan identifikasi server yang sedang memberikan respons kepada client.

4.7 Konfigurasi HAProxy

Tahap berikutnya adalah melakukan konfigurasi HAProxy sebagai load balancer yang bertugas mendistribusikan permintaan dari client menuju kedua backend server. Konfigurasi dilakukan dengan mengubah file `/etc/haproxy/haproxy.cfg`. Pada implementasi ini digunakan metode Round Robin, sehingga setiap permintaan HTTP akan dialihkan secara bergantian ke Web Server 1 dan Web Server 2.

```
backend webservers
    balance roundrobin
    option httpchk GET /
    server ws1 192.168.100.18:80 check
    server ws2 192.168.100.19:80 check
```

Konfigurasi tersebut mendefinisikan dua buah server backend yang akan menerima permintaan dari client. Parameter `balance roundrobin` digunakan untuk mendistribusikan permintaan secara bergantian ke setiap backend. Sementara itu, parameter `option httpchk GET /` mengaktifkan mekanisme *health check* dengan mengirimkan permintaan HTTP secara berkala ke halaman utama masing-masing server. Apabila salah satu server tidak memberikan respons, HAProxy akan menandainya sebagai *DOWN* sehingga tidak lagi menerima permintaan baru.

```

GNU nano 6.2 /etc/haproxy/haproxy.cfg
crt-base /etc/ssl/private

# See: https://ssl-config.mozilla.org/#server=haproxy&server-version=2.0.3&config=intermedi
ssl-default-bind-ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-EC
ssl-default-bind-ciphersuites TLS_AES_128_GCM_SHA256:TLS_AES_256_GCM_SHA384:TLS_CHACHA20_P
ssl-default-bind-options ssl-min-ver TLSv1.2 no-tls-tickets

defaults
    log global
    mode http
    option httplog
    option dontlognull
    timeout connect 5000
    timeout client 50000
    timeout server 50000
    errorfile 400 /etc/haproxy/errors/400.http
    errorfile 403 /etc/haproxy/errors/403.http
    errorfile 408 /etc/haproxy/errors/408.http
    errorfile 500 /etc/haproxy/errors/500.http
    errorfile 502 /etc/haproxy/errors/502.http
    errorfile 503 /etc/haproxy/errors/503.http
    errorfile 504 /etc/haproxy/errors/504.http

frontend web_front
    bind *:80
    default_backend web_servers

backend web_servers
    balance roundrobin
    option httpchk GET /
    server webserver1 192.168.56.11:80 check
    server webserver2 192.168.56.12:80 check

```

Gambar 11 Konfigurasi file haproxy.cfg

Selain itu, konfigurasi juga memanfaatkan fitur HTTP Health Check menggunakan parameter `option httpchk GET /`. Fitur ini memungkinkan HAProxy melakukan pemeriksaan secara berkala terhadap kondisi masing-masing backend server. Apabila salah satu server tidak memberikan respons, maka server tersebut secara otomatis dikeluarkan dari daftar server aktif sehingga seluruh permintaan akan dialihkan ke server yang masih berjalan.

```

adib@haproxy:~$ sudo systemctl status haproxy
[sudo] password for adib:
• haproxy.service - HAProxy Load Balancer
   Loaded: loaded (/lib/systemd/system/haproxy.service; enabled; vendor preset: enabled)
   Active: active (running) since Fri 2026-06-26 09:06:08 UTC; 3h 1min ago
     Docs: man:haproxy(1)
           file:/usr/share/doc/haproxy/configuration.txt.gz
   Process: 652 ExecStartPre=/usr/sbin/haproxy -Ws -f $CONFIG -c -q $EXTRA_OPTS (code=exited, status=0)
   Main PID: 683 (haproxy)
      Tasks: 2 (limit: 1011)
     Memory: 74.7M
        CPU: 5.642s
   CGroup: /system.slice/haproxy.service
           └─683 /usr/sbin/haproxy -Ws -f /etc/haproxy/haproxy.cfg -p /run/haproxy.pid -S /run/haproxy.sock
             750 /usr/sbin/haproxy -Ws -f /etc/haproxy/haproxy.cfg -p /run/haproxy.pid -S /run/haproxy.sock

Jun 26 09:06:08 haproxy systemd[1]: Started HAProxy Load Balancer.
Jun 26 09:06:09 haproxy haproxy[750]: [WARNING] (750) : Server web_servers/webserver1 is DOWN, reason: connect() to 192.168.56.11:80 failed: Connection refused (errno=111)
Jun 26 09:06:09 haproxy haproxy[750]: [WARNING] (750) : Server web_servers/webserver2 is DOWN, reason: connect() to 192.168.56.12:80 failed: Connection refused (errno=111)
Jun 26 09:06:09 haproxy haproxy[750]: [NOTICE] (750) : haproxy version is 2.4.30-0ubuntu0.22.04.2
Jun 26 09:06:09 haproxy haproxy[750]: [NOTICE] (750) : path to executable is /usr/sbin/haproxy
Jun 26 09:06:09 haproxy haproxy[750]: [ALERT] (750) : backend 'web_servers' has no server available
Jun 26 09:06:39 haproxy haproxy[750]: [WARNING] (750) : Server web_servers/webserver1 is UP, reason: connect() to 192.168.56.11:80 succeeded
Jun 26 09:06:49 haproxy haproxy[750]: [WARNING] (750) : Server web_servers/webserver2 is UP, reason: connect() to 192.168.56.12:80 succeeded
Jun 26 12:02:53 haproxy haproxy[750]: [WARNING] (750) : Server web_servers/webserver1 is DOWN, reason: connect() to 192.168.56.11:80 failed: Connection refused (errno=111)
Jun 26 12:07:22 haproxy haproxy[750]: [WARNING] (750) : Server web_servers/webserver1 is UP, reason: connect() to 192.168.56.11:80 succeeded
lines 1-24/24 (END)

```

Gambar 12 Status layanan HAProxy beserta health check backend

Setelah konfigurasi selesai dilakukan, layanan HAProxy dijalankan kembali menggunakan perintah:

```
sudo systemctl restart haproxy
```

Selanjutnya dilakukan pemeriksaan status layanan menggunakan:

```
sudo systemctl status haproxy
```

Perintah `systemctl status haproxy` digunakan untuk memverifikasi bahwa layanan HAProxy berhasil dijalankan tanpa mengalami kesalahan konfigurasi. Selain status layanan, pemeriksaan ini juga memastikan bahwa seluruh backend berhasil dikenali sebelum dilakukan proses pengujian *load balancing*. Hasil pengujian menunjukkan bahwa HAProxy berhasil berjalan dengan status *active (running)* dan mampu mendeteksi kondisi kedua backend server.

4.8 Pengujian Load Balancing

Setelah konfigurasi HAProxy selesai dilakukan, tahap berikutnya adalah menguji mekanisme distribusi trafik menggunakan algoritma Round Robin. Pengujian dilakukan dengan mengirimkan permintaan HTTP secara berulang menggunakan perintah `curl` menuju alamat IP HAProxy.

```
curl http://192.168.56.10
```

Hasil pengujian menunjukkan bahwa respons yang diterima client berasal secara bergantian dari Web Server 1 dan Web Server 2. Pergantian respons tersebut menunjukkan bahwa HAProxy telah berhasil menerapkan algoritma Round Robin sesuai konfigurasi yang telah dibuat.

Table 9 hasil respon dari beberapa percobaan.

Percobaan	Hasil Respon
1	Web Server 1
2	Web Server 2
3	Web Server 1
4	Web Server 2
5	Web Server 1

Berdasarkan hasil pengujian tersebut, terlihat bahwa setiap permintaan HTTP didistribusikan secara bergantian ke Web Server 1 dan Web Server 2. Pola distribusi ini menunjukkan bahwa algoritma *Round Robin* telah bekerja sesuai konfigurasi. Dengan

4.9 Pengujian Failover

Selain melakukan pengujian distribusi trafik, sistem juga diuji terhadap kemampuan failover. Pengujian dilakukan dengan menghentikan layanan Apache pada salah satu backend server sehingga server tersebut dianggap tidak aktif oleh HAProxy. Ketika salah satu backend server berhenti memberikan layanan, HAProxy secara otomatis mendeteksi kondisi tersebut melalui mekanisme health check dan menghapus server yang bermasalah dari daftar server aktif.

Table 11 Hasil dari pengujian dengan kondisi berbeda.

Kondisi	Hasil
Kedua server aktif	Trafik dibagi merata
Web Server 1 dimatikan	Semua trafik menuju Web Server 2
Web Server 2 diaktifkan kembali	Trafik kembali dibagi merata

Hasil pengujian menunjukkan bahwa HAProxy mampu mendeteksi perubahan kondisi backend secara otomatis melalui mekanisme *health check*. Ketika salah satu web server tidak lagi memberikan respons, HAProxy segera menghentikan pengiriman permintaan ke server tersebut dan mengalihkan seluruh trafik menuju server yang masih aktif. Setelah server kembali aktif, HAProxy secara otomatis memasukkannya kembali ke dalam kelompok backend sehingga distribusi trafik kembali berlangsung secara normal.

Selanjutnya dilakukan kembali pengujian menggunakan perintah curl menuju alamat IP HAProxy. Hasil pengujian menunjukkan bahwa seluruh permintaan dialihkan menuju backend server yang masih aktif tanpa memerlukan perubahan konfigurasi maupun intervensi administrator.

```

System load: 0.0          Processes:           100
Usage of /:  33.1% of 9.74GB    Users logged in:   0
Memory usage: 29%          IPv4 address for enp0s3: 192.168.56.10
Swap usage:  0%

Expanded Security Maintenance for Applications is not enabled.

238 updates can be applied immediately.
178 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection
or proxy settings

Last login: Fri Jun 26 08:25:43 UTC 2026 on tty1
adib@haproxy:~$ curl http://192.168.56.10
<h1>WEB SERVER 1 - kelompok 7 UMR4H<h1>
adib@haproxy:~$ curl http://192.168.56.10
<h1>WEB SERVER 2 - kelompok 7 UMR4H<h1>
adib@haproxy:~$ curl http://192.168.56.10
<h1>WEB SERVER 1 - kelompok 7 UMR4H<h1>
adib@haproxy:~$ curl http://192.168.56.10
<h1>WEB SERVER 2 - kelompok 7 UMR4H<h1>
adib@haproxy:~$ curl http://192.168.56.10
<h1>WEB SERVER 1 - kelompok 7 UMR4H<h1>
adib@haproxy:~$ curl http://192.168.56.10
<h1>WEB SERVER 2 - kelompok 7 UMR4H<h1>
adib@haproxy:~$ curl http://192.168.56.10
<h1>WEB SERVER 1 - kelompok 7 UMR4H<h1>
adib@haproxy:~$ curl http://192.168.56.10
<h1>WEB SERVER 2 - kelompok 7 UMR4H<h1>
adib@haproxy:~$ _

```

Gambar 14 Hasil pengujian failover ketika salah satu backend server tidak aktif.

Table 12 Hasil Pengujian Failover

Kondisi Sistem	Hasil
Kedua backend aktif	Trafik terbagi merata
Web Server 1 dihentikan	Semua request dialihkan ke Web Server 2
Web Server 1 dijalankan kembali	Distribusi kembali normal
Web Server 2 dihentikan	Semua request dialihkan ke Web Server 1

Hasil tersebut menunjukkan bahwa mekanisme High Availability telah berhasil diterapkan. Layanan tetap dapat diakses meskipun salah satu web server mengalami gangguan sehingga kontinuitas layanan tetap terjaga.

4.10 Analisis Hasil Implementasi

Table 13 Rekapitulasi Hasil Implementasi

No	Tahapan	Hasil
1	Instalasi Ubuntu Server	Berhasil
2	Konfigurasi IP Address	Berhasil
3	Integrasi GNS3	Berhasil
4	Pengujian Ping	Berhasil
5	Instalasi Apache2	Berhasil

6	Konfigurasi HAProxy	Berhasil
7	Pengujian Load Balancing	Berhasil
8	Pengujian Failover	Berhasil

Table 14 Analisis Kesesuaian dengan Tujuan Proyek

Tujuan	Hasil Implementasi
Membangun High Availability	Tercapai
Mendistribusikan trafik	Tercapai
Implementasi Round Robin	Tercapai
Health Check	Tercapai
Failover Otomatis	Tercapai

Berdasarkan seluruh tahapan implementasi dan pengujian yang telah dilakukan, sistem *High Availability Server* berhasil dibangun sesuai dengan rancangan yang telah disusun pada Bab III. Seluruh komponen sistem, mulai dari virtual machine, konfigurasi jaringan, layanan Apache2, hingga HAProxy sebagai *load balancer*, dapat diintegrasikan dengan baik sehingga membentuk suatu infrastruktur yang mampu memberikan layanan web secara berkesinambungan.

Pada tahap implementasi jaringan, seluruh perangkat berhasil dikonfigurasi menggunakan alamat IP statis sesuai dengan *IP Address Plan*. Hasil pengujian konektivitas menggunakan perintah *ping*, pemeriksaan *routing table*, dan *traceroute* menunjukkan bahwa seluruh perangkat dapat saling berkomunikasi tanpa mengalami *packet loss*. Hal tersebut membuktikan bahwa konfigurasi jaringan dan *static routing* telah diterapkan dengan benar sehingga mampu menjadi fondasi bagi implementasi layanan pada tahap berikutnya.

Implementasi Apache2 pada kedua backend server juga berhasil dilakukan dengan baik. Hasil pemeriksaan menggunakan perintah *systemctl status apache2* menunjukkan bahwa layanan web berada pada status *active (running)*. Selain itu, masing-masing web server menampilkan halaman identitas yang berbeda sehingga proses identifikasi backend selama pengujian dapat dilakukan dengan mudah. Kondisi tersebut menunjukkan bahwa kedua backend telah siap menerima permintaan yang diteruskan oleh HAProxy.

Konfigurasi HAProxy sebagai *load balancer* berhasil diterapkan menggunakan algoritma *Round Robin*. Berdasarkan hasil pengujian menggunakan perintah *curl*, respons HTTP diterima secara bergantian dari Web Server 1 dan Web Server 2. Hasil tersebut menunjukkan bahwa proses distribusi trafik berjalan sesuai konfigurasi sehingga beban

permintaan tidak terpusat pada satu server saja. Dengan distribusi beban yang lebih merata, pemanfaatan sumber daya server menjadi lebih optimal dan potensi penurunan performa akibat penumpukan trafik dapat diminimalkan.

Selain mekanisme *load balancing*, pengujian *failover* menunjukkan bahwa HAProxy mampu mendeteksi kegagalan salah satu backend melalui mekanisme *health check*. Ketika layanan Apache2 pada salah satu web server dihentikan, HAProxy secara otomatis menghapus server tersebut dari daftar backend aktif dan mengalihkan seluruh permintaan ke server lain yang masih berjalan. Setelah server kembali aktif, HAProxy secara otomatis memasukkannya kembali ke dalam kelompok backend tanpa memerlukan perubahan konfigurasi. Hasil ini membuktikan bahwa mekanisme *High Availability* telah berjalan sesuai dengan tujuan implementasi.

Secara keseluruhan, implementasi sistem berhasil memenuhi tujuan proyek, yaitu membangun layanan web yang memiliki tingkat ketersediaan lebih baik dibandingkan arsitektur *single server*. Melalui penerapan HAProxy sebagai *load balancer*, sistem mampu mendistribusikan trafik secara merata sekaligus menjaga kontinuitas layanan ketika salah satu backend mengalami gangguan.

4.11 Evaluasi Sistem

Table 15 Evaluasi Sistem

Aspek	Keterangan
Ketersediaan layanan	Sangat Baik
Distribusi trafik	Berjalan sesuai Round Robin
Failover	Berjalan otomatis
Kemudahan implementasi	Baik
Skalabilitas	Baik
Keamanan	Perlu pengembangan

Table 16 Evaluasi Implementasi

Kelebihan	Keterbatasan
Open source	Belum HTTPS
Failover otomatis	Hanya dua backend
Konfigurasi sederhana	Belum ada monitoring

Distribusi trafik merata	Belum ada database replication
--------------------------	--------------------------------

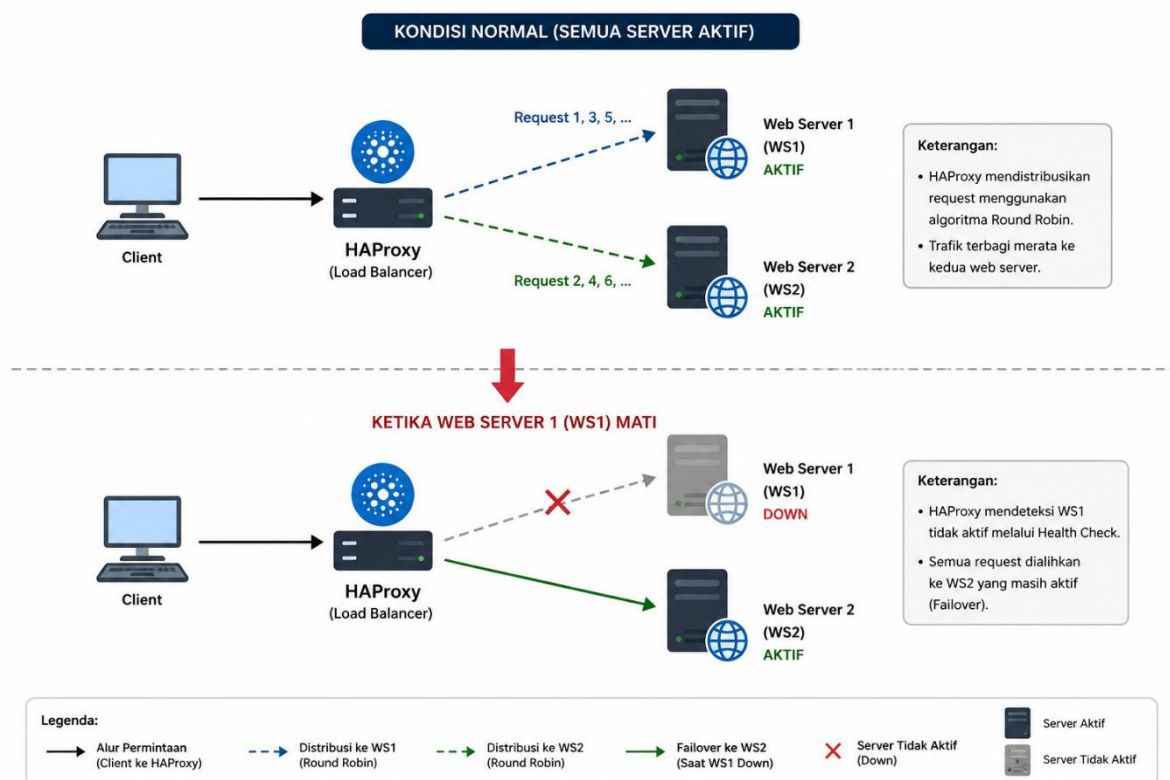
Berdasarkan hasil implementasi dan pengujian yang telah dilakukan, sistem *High Availability Server* yang dibangun telah memenuhi fungsi utama sebagai infrastruktur yang mampu mendistribusikan permintaan pengguna serta mempertahankan ketersediaan layanan ketika terjadi gangguan pada salah satu web server. Seluruh komponen sistem bekerja sesuai dengan perancangan sehingga komunikasi antara client, router, HAProxy, dan kedua backend server dapat berlangsung dengan baik.

Implementasi ini memiliki beberapa keunggulan. Pertama, penggunaan HAProxy sebagai *load balancer* memungkinkan distribusi trafik dilakukan secara otomatis menggunakan algoritma *Round Robin*, sehingga beban kerja dapat dibagi secara lebih merata. Kedua, fitur *health check* yang dimiliki HAProxy mampu memantau kondisi setiap backend secara berkala sehingga proses *failover* dapat dilakukan secara otomatis tanpa memerlukan intervensi administrator. Ketiga, penggunaan perangkat lunak *open source* seperti Ubuntu Server, Apache2, HAProxy, dan GNS3 menjadikan implementasi ini lebih ekonomis, fleksibel, serta mudah dikembangkan sesuai kebutuhan.

Meskipun demikian, sistem yang dibangun masih memiliki beberapa keterbatasan. Implementasi hanya menggunakan dua buah backend server sehingga kapasitas layanan masih terbatas apabila jumlah pengguna meningkat secara signifikan. Selain itu, sistem belum menerapkan mekanisme keamanan seperti HTTPS atau SSL/TLS untuk mengenkripsi komunikasi antara client dan server. Implementasi juga belum dilengkapi dengan *database replication*, sehingga apabila aplikasi yang dijalankan memerlukan basis data bersama, diperlukan mekanisme sinkronisasi tambahan. Dari sisi ketersediaan layanan, HAProxy masih beroperasi sebagai satu *load balancer*, sehingga komponen tersebut masih berpotensi menjadi *Single Point of Failure* apabila mengalami gangguan.

Sebagai pengembangan pada penelitian atau implementasi selanjutnya, sistem dapat ditingkatkan dengan menambahkan jumlah backend server agar kapasitas layanan menjadi lebih besar. Selain itu, implementasi *Keepalived* dapat diterapkan untuk menyediakan *redundancy* pada *load balancer* sehingga tidak lagi bergantung pada satu HAProxy. Penggunaan HTTPS dengan sertifikat SSL/TLS juga perlu diterapkan untuk meningkatkan keamanan komunikasi data. Di samping itu, integrasi dengan sistem *monitoring* seperti Prometheus dan Grafana dapat membantu administrator memantau kondisi server, penggunaan sumber daya, serta performa layanan secara *real-time*.

Dengan berbagai hasil implementasi yang telah dicapai, dapat disimpulkan bahwa sistem *High Availability Server* yang dibangun telah berhasil memenuhi kebutuhan dasar akan layanan web yang lebih andal dan memiliki tingkat ketersediaan yang lebih baik. Meskipun masih terdapat beberapa keterbatasan, sistem ini telah memberikan gambaran yang jelas mengenai penerapan teknologi *High Availability* dan *Load Balancing* pada lingkungan jaringan berbasis Linux.



Gambar 15 Ringkasan Mekanisme High Availability Setelah Implementasi

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan, proyek Implementasi High Availability Server dengan Load Balancing menggunakan HAProxy berhasil direalisasikan sesuai dengan tujuan yang telah ditetapkan. Sistem dibangun menggunakan simulator GNS3 dengan memanfaatkan tiga virtual machine Ubuntu Server, yaitu satu server yang berfungsi sebagai HAProxy *load balancer* dan dua server backend yang menjalankan layanan Apache2.

Konfigurasi jaringan berhasil diterapkan menggunakan pengalamatan IP statis dan *static routing* sehingga seluruh perangkat pada topologi dapat saling berkomunikasi dengan baik. Hasil pengujian konektivitas menunjukkan bahwa setiap perangkat mampu bertukar data tanpa mengalami *packet loss*, sehingga infrastruktur jaringan telah berfungsi sesuai dengan rancangan.

Implementasi HAProxy menggunakan algoritma **Round Robin** berhasil mendistribusikan permintaan (*request*) dari client secara bergantian ke Web Server 1 dan Web Server 2. Hasil pengujian menggunakan perintah *curl* menunjukkan bahwa respons HTTP diterima secara bergantian dari kedua backend server, yang menandakan bahwa mekanisme *load balancing* berjalan dengan baik.

Selain itu, fitur **health check** pada HAProxy mampu mendeteksi kondisi backend server secara otomatis. Ketika salah satu web server dihentikan, HAProxy mengalihkan seluruh trafik menuju server yang masih aktif tanpa memerlukan perubahan konfigurasi maupun intervensi administrator. Setelah server kembali aktif, HAProxy secara otomatis memasukkannya kembali ke dalam kelompok backend sehingga proses distribusi trafik kembali berjalan normal.

Berdasarkan seluruh hasil implementasi dan pengujian tersebut, dapat disimpulkan bahwa sistem yang dibangun telah memenuhi tujuan proyek, yaitu meningkatkan ketersediaan (*availability*) layanan web, mengurangi risiko *Single Point of Failure*, serta mendistribusikan beban layanan secara lebih merata melalui penerapan teknologi *High Availability* dan *Load Balancing*.

5.2 Saran

Berdasarkan hasil implementasi yang telah dilakukan, terdapat beberapa saran yang dapat dijadikan sebagai pengembangan pada penelitian maupun implementasi selanjutnya, antara lain:

1. Menambahkan jumlah backend server sehingga kapasitas layanan dapat meningkat dan sistem mampu menangani jumlah pengguna yang lebih besar.
2. Mengimplementasikan Keepalived atau teknologi serupa untuk menyediakan *redundancy* pada HAProxy sehingga *load balancer* juga memiliki mekanisme *failover* dan tidak menjadi *Single Point of Failure*.
3. Menerapkan protokol HTTPS menggunakan sertifikat SSL/TLS agar komunikasi antara client dan server menjadi lebih aman.
4. Menambahkan sistem pemantauan (*monitoring*) seperti Prometheus dan Grafana untuk memantau kondisi server, penggunaan sumber daya, serta performa layanan secara *real-time*.
5. Mengembangkan implementasi dengan menambahkan *database replication* atau *database clustering* sehingga sistem tidak hanya memiliki *high availability* pada layanan web, tetapi juga pada layanan basis data.
6. Melakukan pengujian performa menggunakan alat seperti Apache Benchmark (*ab*) atau JMeter untuk mengetahui kemampuan sistem dalam menangani beban trafik yang lebih tinggi serta memperoleh data kuantitatif mengenai peningkatan performa yang dihasilkan.

DAFTAR PUSTAKA

- Canonical Ltd. (2024). *Ubuntu Server 22.04 LTS Documentation*. Retrieved from Ubuntu: <https://ubuntu.com/server/docs>
- Forouzan, B. A. (2013). *Data Communication and Networking*. New York: McGraw-Hill Education.
- GNS3 Technologies. (2024). *GNS3 Documentation*. Retrieved from GNS3: <https://docs.gns3.com>
- HAProxy Technologies. (2024). *HAProxy Configuration Manual Version 2.4*. Retrieved from HAProxy: <https://www.haproxy.org/download/2.4/doc/configuration.txt>
- Internet Engineering Task Force. (1981). *RFC 791 — Internet Protocol*. Retrieved from IETF Datatracker: <https://datatracker.ietf.org/doc/html/rfc791>
- Internet Engineering Task Force. (1993). *RFC 1519 — Classless Inter-Domain Routing (CIDR)*. Retrieved from IETF Datatracker: <https://datatracker.ietf.org/doc/html/rfc1519>
- Tanenbaum, A. S., & Wetherall, D. J. (2011). *Computer Networks*. Boston: Person Education.